Systems Literacy

Module 2

How Computers Think and Store Data

Topics To Be Covered

Focus: How physical signals become logical information.

- **The Binary Abstraction:** Moving from high/low voltage to bits, bytes, and words.
- **Encoding Standards:** Beyond binary—ASCII, Unicode (UTF-8), Hexadecimals, and the importance of character encoding in global systems.
- **Data Types & Metadata:** How the OS distinguishes a .jpg from an .exe beyond the extension.
- **File System Architecture:** Journaling file systems (NTFS, APFS, ext4) and how data is physically indexed on a disk.
- **Hands-on:** Using a Hex Editor to view "magic bytes" in different file types; binary-to-decimal mental math.

The Binary Abstraction

The Concept: Computers don't "know" numbers; they know **States**.

- **The Transistor:** Microscopic switch. On (1) or Off (0). This microscopic switch stores a bit (0 or 1) that forms the basis of computers.
- **The Bit:** The smallest unit of information.
- **The Nibble:** 4 bits of data is called a nibble.
- **The Byte:** 8 bits.
  - *why* 8 bits? It was the historical standard for representing a single character and became the "atom" of addressing memory.

Base-2 Math: How 1011 in binary equals 11 in decimal.

- Formula: $1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = 11.$

Character Encoding: ASCII to Unicode

Character encoding is **foundational** for global systems because it ensures that **the same bytes mean the same text everywhere**, across languages, platforms, and time.

ASCII and then Unicode were developed as ways for systems to understand and store letters and symbols.

- **ASCII:** The 7-bit original. Limited to 128 characters (English-centric).
- **Unicode (UTF-8):** The universal standard. UTF-8 is "variable width"—it uses 1 byte for English but can use up to 4 bytes for emojis or Kanji.
- **"Why":** "Mojibake" (garbled text symbols). This happens when a system guesses the wrong encoding standard.

Character Encoding: ASCII to Unicode

ASCII (American Standard Code for Information Interchange) is a character encoding standard that maps numbers to text characters so computers can represent and exchange text.

Key points, succinctly:

- ASCII uses **7 bits**, allowing **128 symbols (0–127)**.
- Each number corresponds to a character (letters, digits, punctuation, or control codes).
- Example: `65 → 'A'`, `97 → 'a'`, `48 → '0'`, `10 → newline`.
- The first 32 codes are **control characters** (e.g., newline, tab), not printable.

It became the foundation for later encodings like **Unicode** (UTF-8 includes ASCII as a subset).
In essence: ASCII is the simplest, original "number ↔ character" mapping that made text processing possible on early computers and networks.

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	|
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

Character Encoding: ASCII to Unicode

UTF-8 is a way to encode **Unicode characters into bytes**, so computers can store and transmit text.

- **Unicode**: a global catalog that assigns every character a unique number (**code point**), e.g.
  - `'A' → U+0041`
  - `'₹' → U+20B9`
  - `'中' → U+4E2D`
- **UTF-8**: a **byte-level encoding** that turns those code points into 1–4 bytes.
- U+ is a label for humans to understand that it represents a Unicode. The code points following the U+ are in hexadecimals.

UTF-8 is **variable-length**:
- Common characters use fewer bytes.
- Rare / complex characters use more bytes.

Unicode answers "What number represents this character?" UTF-8 answers "How do we store that number as bytes?"

Hexadecimals

Hexadecimal (or "hex") is a base-16 number system, using digits 0-9 and letters A-F (representing 10-15), commonly used in computing to represent binary data compactly because one hex digit perfectly represents four binary bits (a nibble). It simplifies reading and writing large binary numbers, like showing colors in HTML (e.g., #FFFFFF for white) or memory addresses, by grouping bits into manageable chunks.

- **Base-16:** Uses 16 unique symbols.
- **Digits:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A (10), B (11), C (12), D (13), E (14), F (15).
- **Compact Binary:** One hex digit = 4 binary digits (bits).

Common Uses

- **Computer Programming:** Representing memory locations, colors (HTML/CSS), and data.
- **Data Representation:** A shorthand for binary, which computers use internally.

HEXADECIMAL NUMBER SYSTEM TABLE

BYJU'S The Learning App

Decimal Numbers	4-bit Binary Number	Hexadecimal Number
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Hexadecimals

How 65 becomes `0x41`
This is **base conversion**, nothing more. **Decimal** → **Binary** → **Hex**

Decimal 65 to Binary

65 = 64 + 1
  = $2^6 + 2^0$
Binary: 01000001

Hex: Group into 4-bit chunks

0100 0001
4 1
Hex: 0x41 (0x is a label for humans to understand that the number is represented in hexadecimals).

Hexadecimals

Why is Hex used everywhere?

- 1 Hex digit = 4 bits
- 1 byte = 2 hex digits
- Much easier than binary, less error-prone than decimal
- Hex is just a **convenient lens** for humans.

Concept	What it is
Unicode	A mapping from characters → numbers
UTF-8	Rules for turning those numbers → bytes
Hex	A notation humans use to write bytes
Memory	Stores **bits**, not hex or decimal

File Anatomy: Extensions vs. Magic Bytes

The Concept: A file is just a stream of bytes or a sequence of 0s and 1s

- **The OS needs a “map”** (encoding, file type info) to interpret it correctly.
- **Example:** A text file vs an image file both store bytes, but you need the right program to read them.

- **File Extensions:** These are just "hints" for the OS (e.g., .png).

- **Abstraction:** A "Folder" is just a special type of file that contains a list of other files.
- Example:
  - `/Documents` folder contains `resume.pdf`, `notes.txt`, `image.png`
  - The OS reads this “special file” to show what’s inside the folder.

File Extensions

File Extensions

- **Idea:** Letters after a dot in a filename are just hints for the OS.
- **Example:**
  - `.txt` → text file
  - `.png`/`.jpeg` → image file
  - `.mp3` → audio file
- **Note:** Changing the extension doesn't change the actual content of the file.

Magic Bytes (File Signatures)

- **Idea:** The first few bytes in a file tell the OS what kind of file it really is, ignoring the extension.
- **Examples:**
  - PDF files always start with `%PDF-`
  - PNG files always start with `89 50 4E 47` (hex for `.PNG`)
  - JPG files start with `FF D8 FF`

File Systems & Hierarchy

The Disk's Address Book: A file system is like an address book for your disk. It keeps track of where every file lives, so the computer can find it quickly.

Analogy: Think of your file system as a **library catalog**: without it, books (files) would be scattered randomly, and you'd never find what you need.

Hierarchy – Root to Subdirectories

Files are organized in a **hierarchy**, like a **family tree**:
1. **Root**: The top-level folder (/ in Linux/Mac, `C:\` in Windows).
2. **Subdirectories/Folders**: Branches under root where related files are grouped.

Analogy: Your root is the **main street**, and subdirectories are the **side streets and houses** where files live.

File System & Hierarchy

Metadata – Info About Files. Metadata generally means data about the data.

Every file has **metadata**, which is like a **file's ID card**:

- Creation date, last modified date
- File size
- Permissions (who can read/write)

Analogy: If a file were a person, metadata would be its **birth certificate, passport, and access card**.

Journaling – Safety Log for Changes

- Modern file systems (like **NTFS, APFS, ext4**) keep a **journal** of changes.
- If the computer **loses power**, the journal helps **prevent corruption** and ensures data is safe.

Analogy: Journaling is like a **diary of renovations in a house**. If the electricity goes out mid-renovation, you can **look at the diary to see what was done and fix it**.

Lab Work: Digital Forensics

Task A: The Magic Byte Reveal

- Take a simple .jpg image.
- Rename the extension to .txt. Try to open it. (It fails/shows gibberish).
- Use an online Hex Editor (or a CLI tool like xxd) to look at the first 4 bytes.
- Identify the signature (e.g., FF D8 FF E0). This proves the "Identity" of data is inside the file, not the name.

Task B: Calculating Capacity

- ☐ If a text file uses UTF-8 and contains 1,000 characters, how many bytes is it (roughly)?
- ☐ If you have a 1GB drive, why does the OS say you only have ~931MB? (Explain the difference between Decimal 10^9 and Binary 2^{30}—the "Gigabyte vs. Gibibyte" marketing trick).